

[Change page](#) >

Introduction to smart contracts

Page last update: February 12, 2025



+13

[See contributors](#)

On this page



What is a smart contract?

A "smart contract" is simply a program that runs on the Ethereum blockchain. It's a collection of code (its functions) and data (its state) that resides at a specific address on the Ethereum blockchain.

Smart contracts are a type of [Ethereum account](#). This means they have a balance and can be the target of transactions. However they're not controlled by a user, instead they are deployed to the network and run as programmed. User accounts can then interact with a smart contract by submitting transactions that execute a function defined on the smart contract. Smart contracts can define rules, like a regular contract, and automatically enforce them via the code. Smart contracts cannot be deleted by default, and interactions with them are irreversible.

Prerequisites

If you're just getting started or looking for a less technical introduction, we recommend our [introduction to smart contracts](#).

Make sure you've read up on [accounts](#), [transactions](#) and the [Ethereum virtual machine](#) before jumping into the world of smart contracts.

A digital vending machine

Perhaps the best metaphor for a smart contract is a vending machine, as described by [Nick Szabo](#) [↗](#). With the right inputs, a certain output is guaranteed.

To get a snack from a vending machine:

```
1  money + snack selection = snack dispensed
```

This logic is programmed into the vending machine.

A smart contract, like a vending machine, has logic programmed into it. Here's a simple example of how this vending machine would look if it were a smart contract written in Solidity:

```
24      function purchase(uint amount) public payable {
25          require(msg.value >= amount * 1 ether, "Not enough ether");
26          require(cupcakeBalances[address(this)] >= amount, "Not enough cupcakes");
27          cupcakeBalances[address(this)] -= amount;
28          cupcakeBalances[msg.sender] += amount;
29      }
30  }
```

Show all Copy 

Like how a vending machine removes the need for a vendor employee, smart contracts can replace intermediaries in many industries.

Permissionless

Anyone can write a smart contract and deploy it to the network. You just need to learn how to code in a [smart contract language](#), and have enough ETH to deploy your contract. Deploying a smart contract is technically a transaction, so you need to pay [gas](#) in the same way you need to pay gas for a simple ETH transfer. However, gas costs for contract deployment are far higher.

Ethereum has developer-friendly languages for writing smart contracts:

- Solidity
- Vyper

[More on languages](#)

However, they must be compiled before they can be deployed so that Ethereum's virtual machine can interpret and store the contract. [More on compilation](#)

Composability

Smart contracts are public on Ethereum and can be thought of as open APIs. This means you can call other smart contracts in your own smart contract to greatly extend what's possible. Contracts can even deploy other contracts.

Learn more about [smart contract composability](#).

Limitations

Smart contracts alone cannot get information about "real-world" events because they can't retrieve data from offchain sources. This means they can't respond to events in the real world. This is by design. Relying on external information could jeopardise consensus, which is important for security and decentralization.

However, it is important for blockchain applications to be able to use offchain data. The solution is [oracles](#) which are tools that ingest offchain data and make it available to smart

contracts.

Another limitation of smart contracts is the maximum contract size. A smart contract can be a maximum of 24KB or it will run out of gas. This can be circumnavigated by using [The Diamond Pattern](#) .

Multisig contracts

Multisig (multiple-signature) contracts are smart contract accounts that require multiple valid signatures to execute a transaction. This is very useful for avoiding single points of failure for contracts holding substantial amounts of ether or other tokens. Multisigs also divide responsibility for contract execution and key management between multiple parties and prevent the loss of a single private key leading to irreversible loss of funds. For these reasons, multisig contracts can be used for simple DAO governance. Multisigs require N signatures out of M possible acceptable signatures (where $N \leq M$, and $M > 1$) in order to execute. $N = 3$, $M = 5$ and $N = 4$, $M = 7$ are commonly used. A 4/7 multisig requires four out of seven possible valid signatures. This means the funds are still retrievable even if three signatures are lost. In this case, it also means that the majority of key-holders must agree and sign in order for the contract to execute.

Smart contract resources

OpenZeppelin Contracts - *Library for secure smart contract development.*

- openzeppelin.com/contracts/ 
- [GitHub](#) 
- [Community Forum](#) 

Further reading

- [Coinbase: What is a smart contract?](#) 
- [Chainlink: What is a smart contract?](#) 

- [Video: Simply Explained - Smart Contracts](#) ↗
- [Cyfrin Updraft: Web3 learning and auditing platform](#) ↗

[Back to top](#) ↑

Thank you for your feedback!

Make this page even better by answering a few questions. If you need help, you can reach out to the community on our [Discord](#) ↗.

[Open short survey](#)

[Next](#) >
Smart contract languages

< [Previous](#)
Intro to the stack

Website last updated: February 5, 2026

[^ Go to top](#)

Learn

Learn Hub

What is Ethereum?

What is ether (ETH)?

Ethereum wallets

Use

Guides

Choose your wallet

Get ETH

Application explorer

[What is Web3?](#)[Smart contracts](#)[Gas fees](#)[Run a node](#)[Ethereum security and scam prevention](#)[Quiz Hub](#)[Ethereum glossary](#)[Stablecoins](#)[NFTs - Non-fungible tokens](#)[DeFi - Decentralized finance](#)[DAOs - Decentralized autonomous organizations](#)[Decentralized identity](#)[Stake ETH](#)[Layer 2](#)

Build

[Builder's home](#)[Tutorials](#)[Documentation](#)[Start building](#)[Learn Ethereum development](#)[Grants](#)[Foundational topics](#)[UX/UI design fundamentals](#)[Enterprise - Mainnet Ethereum](#)[Founders](#)

Participate

[Community hub](#)[Online communities](#)[Ethereum events](#)[Contributing to ethereum.org](#)[Translation Program](#)[Ethereum bug bounty program](#)[Ethereum Foundation](#)[Ethereum Foundation Blog](#)[Ecosystem Support Program](#)[Devcon](#)

Research

[Ethereum Whitepaper](#)[Ethereum roadmap](#)[Improved security](#)[Technical history of Ethereum](#)

[Open research](#)

[Ethereum Improvement Proposals](#)

[Ethereum governance](#)

[Trillion dollar security project](#)



[About us](#)

[Ethereum brand assets](#)

[Code of conduct](#)

[Jobs](#)

[Privacy policy](#)

[Terms of use](#)

[Cookie policy](#)

[✉ Press Contact](#)